Agent, Model, MCP, and Tooling Interview Guide

1. The Big Idea

This project is an enterprise-style AI assistant hub. A user sends a request from a web UI, a backend agent coordinates the request, a model reasons about what the request means, and MCP-connected tools perform real actions when needed.

The core idea is that an LLM alone is not enough for production systems. Real applications need:

â¢ a frontend for interaction
â¢ a backend for orchestration
â¢ an agent loop for reasoning and control flow
â¢ tools for external actions
â¢ MCP servers for standard tool access
â¢ guardrails for approvals and safety
â¢ observability for tracing and debugging

2. The Four Main Concepts

Model

The model is the reasoning engine.

It is good at:

â¢ understanding natural language
â¢ inferring user intent
â¢ deciding whether a tool is needed
â¢ generating final answers

It is not responsible for directly reading files, updating systems, or enforcing runtime safety.

Agent

The agent is the workflow manager around the model.

It is responsible for:

â¢ receiving the user request from the backend
â¢ collecting conversation history
â¢ attaching tool definitions and policy metadata
â¢ calling the model
â¢ interpreting the model’s response
â¢ executing tool calls through MCP
â¢ asking for human approval when required
â¢ looping until a final answer is ready

The agent does not do deep semantic reasoning by itself. It manages the process that lets the model reason and lets tools execute safely.

MCP Server

An MCP server is a tool provider.

It exposes capabilities using a standard protocol. For example, an MCP server can offer tools such as:

â¢ read_file
â¢ write_file
â¢ list_directory
â¢ search_docs
â¢ query_database

The MCP server does not usually decide when a tool should be used. It simply advertises available tools and executes them when called.

Tool

A tool is a specific action exposed by an MCP server.

Examples:

â¢ read a file
â¢ write a file
â¢ delete a file
â¢ fetch data from an API
â¢ search internal knowledge

In simple terms:

â¢ MCP server = toolbox provider
â¢ tool = one tool inside the toolbox
â¢ agent = coordinator using the toolbox
â¢ model = reasoning system helping the coordinator decide

3. The Most Important Distinction

People often confuse the model and the agent.

The model is not the whole agent.

The model reasons.
The agent manages.

That means:

â¢ the model interprets what the user wants
â¢ the agent turns that interpretation into a safe, executable workflow

This is why the correct high-level flow is:

User -> Frontend -> Backend -> Agent -> Model -> Agent -> MCP Server -> Tool -> Result -> Agent -> Model -> Agent -> Backend -> Frontend -> User

This version is more accurate than placing the model before the agent as a separate top-level stage, because the model lives inside the agent loop.

4. What the Agent Knows Before the Model Runs

Suppose the user says:

Please create a summary from sales.txt.

Before the model reasons about that sentence, the agent can already know several system-level facts:

â¢ this is the latest user message
â¢ it belongs to conversation abc123
â¢ prior chat history exists
â¢ these MCP servers are connected
â¢ these tools are currently available
â¢ some tools are read-only
â¢ some tools require approval for write or delete actions
â¢ the configured model is available for the next turn

This is not deep semantic understanding. The agent is not yet deciding what the sentence means. It is only gathering the context that the model will need.

5. What the Model Figures Out

When the agent sends the request to the model, the model reasons over:

â¢ the user message
â¢ the conversation history
â¢ the available tool definitions
â¢ the policies and instructions

For the message Please create a summary from sales.txt, the model may infer:

â¢ the user wants a summary
â¢ to produce a summary, the file content must be read first
â¢ the read_file tool is likely the right next step
â¢ after the file content is available, the model can summarize it

This is where semantic interpretation happens.

6. Step-by-Step End-to-End Workflow

Scenario: Read and Summarize a File

User prompt:

Please create a summary from sales.txt.

Step 1. User enters a message in the frontend.

The React app sends the text, conversation ID, and selected context to the backend.

Step 2. Backend hands the request to the agent service.

The backend does transport and orchestration. It does not decide the final answer by itself.

Step 3. Agent collects context.

The agent gathers:

â¢ the new user message
â¢ the prior conversation
â¢ connected MCP servers
â¢ the current tool registry
â¢ policy rules such as approval requirements

Step 4. Agent calls the model.

The agent sends the raw user message plus context and tool schemas.

Step 5. Model interprets intent.

The model decides whether:

â¢ it can answer directly, or
â¢ it needs one or more tools

For this example, it likely chooses read_file.

Step 6. Agent receives the model output.

If the model requests a tool call, the agent parses the tool name and arguments.

Step 7. Agent checks safety and execution policy.

If the tool is read-only, execution usually continues immediately.

Step 8. Agent calls the MCP client layer.

The MCP client finds which MCP server owns the requested tool and invokes it using stdio, SSE, or HTTP, depending on the server type.

Step 9. MCP server executes the tool.

For a filesystem server, it opens the file and returns the content.

Step 10. Agent receives the tool result.

The result is now grounded in real system state rather than model guesswork.

Step 11. Agent calls the model again.

This time the model sees the file content and can produce the requested summary.

Step 12. Agent streams the final answer back through the backend to the frontend.

The user sees a live response.

7. Write/Delete Flow and Human-in-the-Loop

Now suppose the user says:

Create a file named todo.txt and add today’s tasks.

The early workflow is the same:

â¢ user sends a message
â¢ agent collects context
â¢ model decides a write tool is needed

The key difference is policy enforcement.

If the requested tool has write or delete implications:

â¢ the agent does not execute it immediately
â¢ the backend creates a pending approval state
â¢ the frontend opens a confirmation modal
â¢ the user approves or rejects the action

If approved:

â¢ the tool runs
â¢ the result goes back to the model
â¢ the model produces the final response

If rejected:

â¢ the tool does not run
â¢ the model can explain that the action was not allowed

This pattern is called HITL, or Human In The Loop.

8. Why the Agent Does Not Need to "Understand" the Whole Paragraph

This is a common interview question and a common conceptual confusion.

The agent does not have to semantically understand the user prompt the way the model

does.

The agent only needs enough system-level understanding to:

â¢ know this is a user message
â¢ associate it with a conversation
â¢ gather memory
â¢ gather tool metadata
â¢ gather policy metadata
â¢ call the model with the right context

The model is the component that interprets what the prompt means.

A useful analogy:

â¢ the agent is like a skilled operations coordinator
â¢ the model is like the subject-matter reasoner

The coordinator can route the work correctly without personally solving the user problem.

9. Why MCP Matters

Without MCP, every tool integration would need custom backend code.

That becomes difficult to scale because each new tool would require:

â¢ its own invocation contract
â¢ its own schema mapping
â¢ its own transport implementation
â¢ its own connection and lifecycle handling

MCP solves this by standardizing:

â¢ tool discovery
â¢ tool schemas
â¢ invocation format

â¢ server connection behavior

That makes tools pluggable instead of hardwired.

10. Why Observability Matters

Production agent systems are hard to debug if you cannot see what happened.

This project includes structured logging and tracing so that teams can inspect:

â¢ the incoming request
â¢ the model call
â¢ the selected tool
â¢ approval checkpoints
â¢ the MCP server execution
â¢ the final response

In interviews, this matters because mature AI systems are not judged only by whether they "work once." They are judged by whether they are safe, debuggable, observable, and maintainable.

11. Likely Interview Questions and Strong Answers

Q1. What is the difference between an agent and a model?

Answer:

The model performs semantic reasoning over language and tool descriptions. The agent is the orchestration layer around the model. It manages context, policy, tool execution, retries, approvals, and the control loop that turns model outputs into real actions.

Q2. What is the difference between an MCP server and a tool?

Answer:

An MCP server is the provider that exposes one or more tools through a standard protocol. A tool is a single callable capability such as reading a file or querying a

service.

Q3. Why not let the model call tools directly?

Answer:

Because production systems need governance. The agent layer validates tool availability, applies policy, routes requests to the correct server, enforces human approval for risky actions, and maintains auditability.

Q4. Why is Human In The Loop important?

Answer:

Because some tool calls can change external state. Write and delete actions should often require user approval so the assistant remains useful without becoming unsafe or uncontrollable.

Q5. Why is strict typing important in agent systems?

Answer:

Agent systems pass many structured payloads between frontend, backend, models, and tools. Strong typing reduces ambiguity, makes failures easier to detect, and helps keep tool contracts reliable across the stack.

12. Best Short Summary for Interviews

If you need one concise summary:

This system is an agentic assistant platform where the backend agent orchestrates a model and a set of MCP-exposed tools. The model interprets user intent, the agent manages workflow and policy, MCP servers expose tool capabilities, and human approval protects risky actions. The result is a safer and more production-ready AI architecture than a simple chat application.